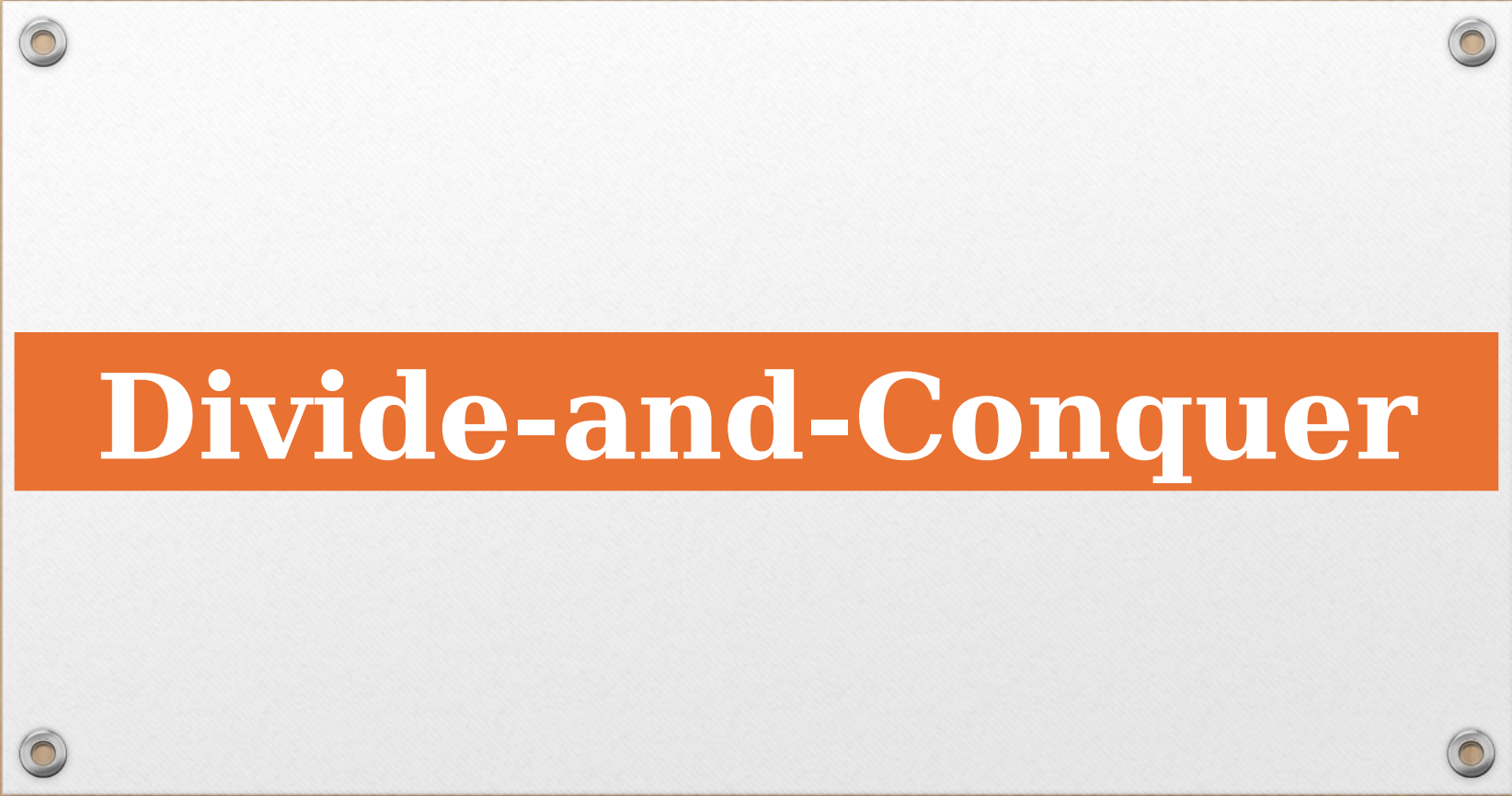




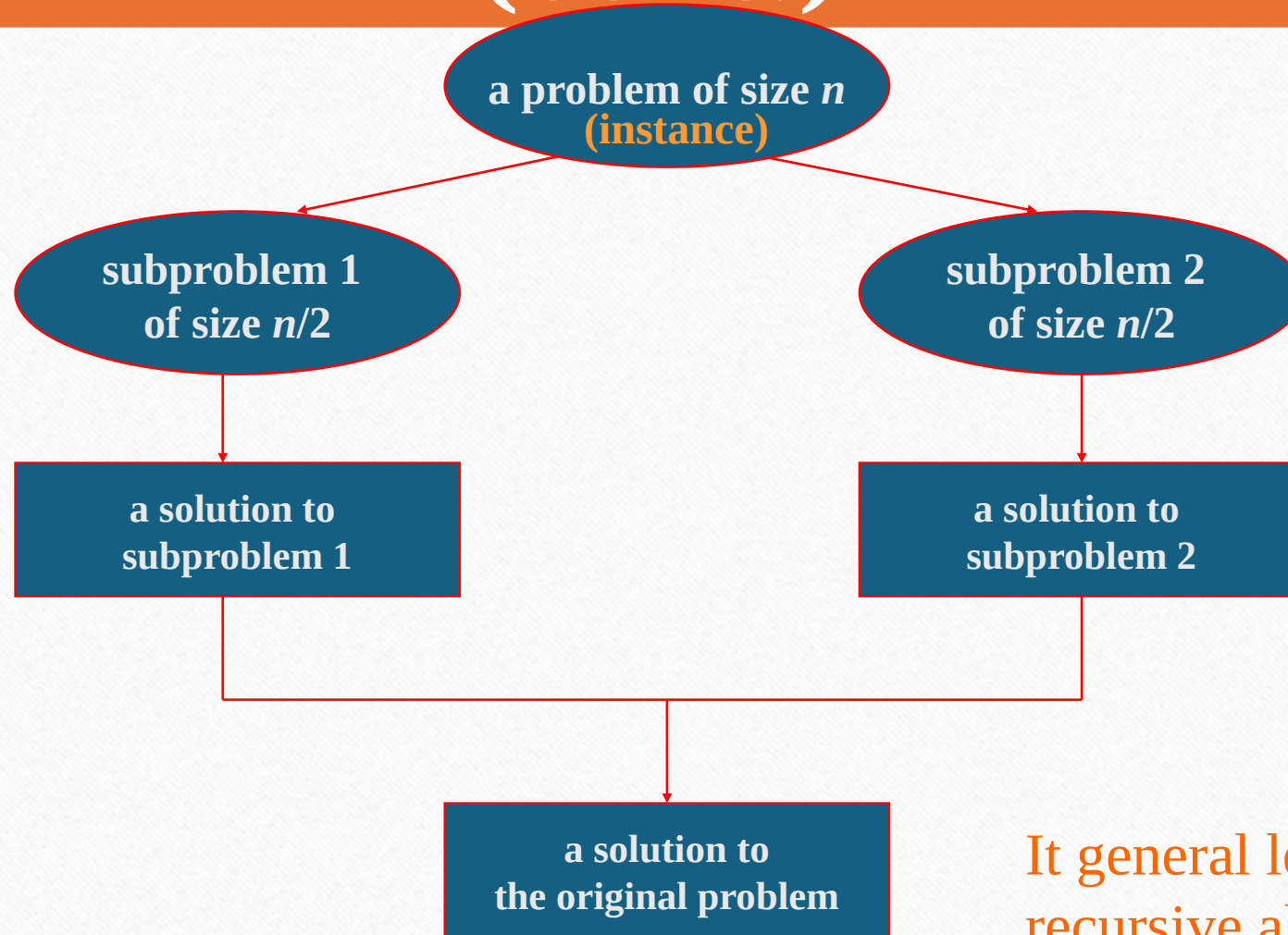
Divide-and-Conquer

Divide-and-Conquer Technique

The most-well known algorithm design strategy:

1. **DIVIDE**: Divide instance of problem into two or more smaller instances
2. **RECUR**: Solve smaller instances recursively (Solve the sub-problem recursively)
3. **CONQUER**: Obtain solution to original (larger) instance by combining these solutions

Divide-and-Conquer Technique (cont.)



It general leads to a recursive algorithm!

General divide & conquer recurrence:

- ▶ An instance of size **n** can be divided into **b** instances of size n/b , with **a** of them needing to be solved. [$a \geq 1, b \geq 1$]
- ▶ **Assume size n is a power of b.** The recurrence for the running time $T(n)$ is as follows:

$$T(n) = aT(n/b) + f(n)$$

where:

$f(n)$ a function that accounts for the time spent on dividing the problem into smaller ones and on combining their solutions

Therefore, the order of growth of $T(n)$ depends on the values of the constants a & b and the order of growth of the function $f(n)$.

Solving recurrence relation

- Substitution Method
- Masters Theorem

Masters Theorem

Theorem: If $f(n) \in \Theta(n^d)$ with $d \geq 0$ in recurrence equation
 $T(n) = aT(n/b) + f(n)$,

then

$$T(n) = \begin{cases} \Theta(n^d) & \text{if } a < b^d \\ \Theta(n^d \log n) & \text{if } a = b^d \\ \Theta(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

Problem 1:

$$T(n) = 4T(n/2) + n$$

$$T(1)=1 \quad a=4 \quad b=2 \quad d=1 \quad (\text{as } f(n)=n)$$

Substitution Method: Represent $n = 2^k$

$$T(n) = 4T(n/2) + n$$

$$4T(2^{k-1}) + 2^k$$

$$= 4(4T(2^{k-2}) + 2^{k-1}) + 2^k$$

$$= 4^2 T(2^{k-2}) + 4 * 2^{k-1} + 2^k$$

$$= 4^2 * (4T(2^{k-3}) + 2^{k-2}) + 4 * 2^{k-1} + 2^k$$

$$= 4^3 * T(2^{k-3}) + 4^2 2^{k-2} + 4 * 2^{k-1} + 2^k$$

$$= 2^{2*3} T(2^{k-3}) + 2^{(2*2)2^{k-2}} + 2^2 2^{k-1} + 2^k$$

$$= 2^{2*3} T(2^{k-3}) + 2^{2*2+k-2} + 2^{2+k-1} + 2^k$$

$$= 2^{2*3} T(2^{k-3}) + 2^{k+2} + 2^{k+1} + 2^k$$

.

.

$$= 2^{2*k} T(2^{k-k}) + 2^{k+k-1} + \dots + 2^{k+2} + 2^{k+1} + 2^k$$

$$= 2^{2*k} T(1) + 2^{k+k-1} + \dots + 2^{k+2} + 2^{k+1} + 2^k$$

$$= 2^{2*k} + 2^{k+k-1} + \dots + 2^{k+2} + 2^{k+1} + 2^k$$

Problem 1:

$$T(n) = 4T(n/2) + n$$

$$T(1)=1 \quad a=4 \quad b=2 \quad d=1 \quad (\text{as } f(n)=n)$$

Highest exponent is 2^{2k}

as $n = 2^k$

Hence $2^{2k} = n^2$

$$T(n) = \theta(n^2)$$

Problem 1:

$$T(n) = 4T(n/2) + n$$

$$T(1)=1 \quad a=4 \quad b=2 \quad d=1 \quad (\text{as } f(n)=n)$$

Highest exponent is 2^{2k}

as $n = 2^k$

Hence $2^{2k} = n^2$

$$T(n) = \theta(n^2)$$

Masters Method:

$$a=4$$

$$b^d = 2^1 = 2$$

$$a > b^d$$

$$\text{Hence } T(n) \in \theta(n^{\log_b a}) = \theta(n^{\log_2 4}) = \theta(n^2)$$

Problem 2:

$$T(n) = 4T(n/2) + n^2 \quad T(1)=1 \quad a=4 \quad b=2 \quad d=2 \text{ (as } f(n) = n^2 \text{)}$$

Masters Method:

$$a=4$$

$$b^d = 2^2 = 4$$

$$a = b^d$$

$$\text{Hence } T(n) \in \theta(n^d \log_b n) = \theta(n^2 \log_2 n)$$

Problem 3:

$$T(n) = 4T(n/2) + n^3$$

$$T(1)=1 \quad a=4 \quad b=2 \quad d=3 \quad (\text{as } f(n)=n^3)$$

Substitution Method: Represent $n = 2^k$

$$\begin{aligned} T(n) &= 4T(n/2) + n^3 = 4T(2^{k-1}) + 2^{3k} \\ &= 4(4T(2^{k-2}) + 2^{3(k-1)}) + 2^{3k} \\ &= 4^2 T(2^{k-2}) + 4 * 2^{3*(k-1)} + 2^{3k} \\ &= 4^2 T(2^{k-2}) + 2^{2+3k-3} + 2^{3k} \\ &= 4^2 T(2^{k-2}) + 2^{3k-1} + 2^{3k} \\ &= 4^2 * (4T(2^{k-3}) + 2^{3*(k-2)}) + 2^{3k-1} + 2^{3k} \\ &= 4^3 * T(2^{k-3}) + 4^2 2^{3*k-3*2} + 2^{3k-1} + 2^{3k} \\ &= 2^{2*3} T(2^{k-3}) + 2^{(2*2)+3k-3*2} + 2^{3k-1} + 2^{3k} \\ &= 2^{2*3} T(2^{k-3}) + 2^{3k-2} + 2^{3k-1} + 2^{3k} \\ &\quad \vdots \\ &= 2^{2*k} T(2^{k-k}) + 2^{3k-(k-2)} + \dots + 2^{3k-2} + 2^{3k-1} + 2^{3k} \\ &= 2^{2*k} T(1) + 2^{3k-(k-2)} + \dots + 2^{3k-2} + 2^{3k-1} + 2^{3k} \end{aligned}$$

Problem 3:

$$T(n) = 4T(n/2) + n^3$$
$$T(1)=1 \quad a=4 \quad b=2 \quad d=3 \text{ (as } f(n) = n^3 \text{)}$$

Highest exponent is 2^{3k}
as $n = 2^k$, $2^{3k} = n^3$
 $T(n) = \theta(n^3)$

Masters Method:

$$a=4$$

$$b^d = 2^3 = 8$$

$$a < b^d$$

$$\text{Hence } T(n) \in \theta(n^d) = \theta(n^3)$$

Divide-and-Conquer Examples

- ☼ Merge sort
- ▢ Quicksort
- ▢ Binary search
- ▢ Binary Tree Traversals and Related Properties
- ▢ Matrix multiplication: Strassen's algorithm
- ▢ Multiplication of large integers

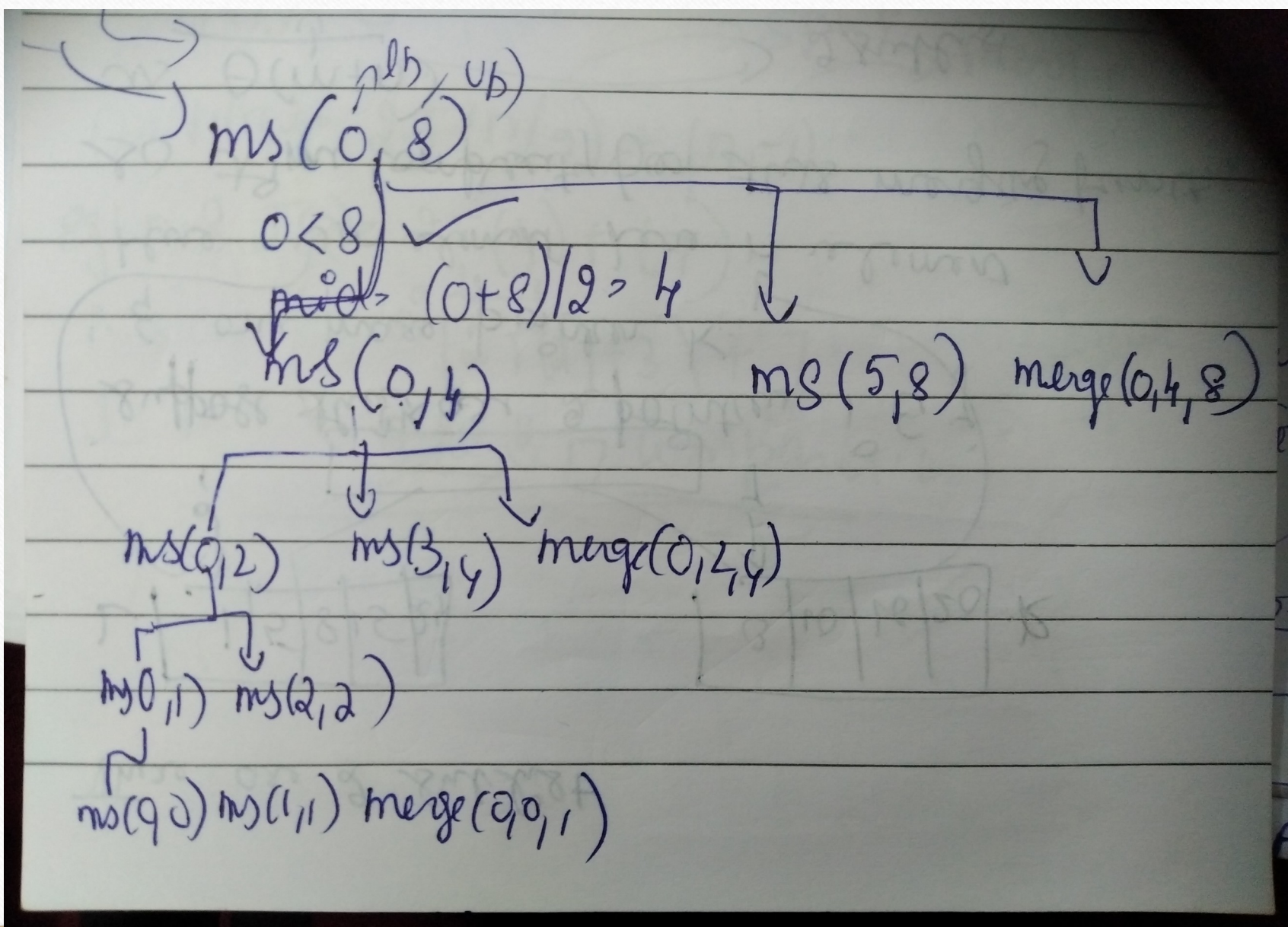


Mergesort

- **Split array $A[0..n-1]$ into about equal halves and make copies of each half in arrays B and C**
- **Sort arrays B and C recursively**
- **Merge sorted arrays B and C into array A as follows:**
 - **Repeat the following until no elements remain in one of the arrays:**
 - **compare the first elements in the remaining unprocessed portions of the arrays**
 - **copy the smaller of the two into A, while incrementing the index indicating the unprocessed portion of that array**
 - **Once all elements in one of the arrays are processed, copy the remaining unprocessed elements from the other array into A.**

15	5	24	8	1	3	16	10	20
----	---	----	---	---	---	----	----	----

```
Mergesort(A, LB, UB)  
{  
  If(LB < UB)  
  {  
    MID = (LB + UB) / 2  
    Mergesort(A, LB, MID)  
    Mergesort(A, MID + 1, UB)  
    Merge(A, LB, MID, MID + 1, UB)  
  }  
}
```



Pseudocode of Mergesort

ALGORITHM *Mergesort*($A[0..n - 1]$)

//Sorts array $A[0..n - 1]$ by recursive mergesort

//Input: An array $A[0..n - 1]$ of orderable elements

//Output: Array $A[0..n - 1]$ sorted in nondecreasing order

if $n > 1$

 copy $A[0..\lfloor n/2 \rfloor - 1]$ to $B[0..\lfloor n/2 \rfloor - 1]$

 copy $A[\lfloor n/2 \rfloor..n - 1]$ to $C[0..\lceil n/2 \rceil - 1]$

Mergesort($B[0..\lfloor n/2 \rfloor - 1]$)

Mergesort($C[0..\lceil n/2 \rceil - 1]$)

Merge(B, C, A)

Pseudocode of Merge

ALGORITHM *Merge*($B[0..p-1]$, $C[0..q-1]$, $A[0..p+q-1]$)

//Merges two sorted arrays into one sorted array

//Input: Arrays $B[0..p-1]$ and $C[0..q-1]$ both sorted

//Output: Sorted array $A[0..p+q-1]$ of the elements of B and C

$i \leftarrow 0$; $j \leftarrow 0$; $k \leftarrow 0$

while $i < p$ **and** $j < q$ **do**

if $B[i] \leq C[j]$

$A[k] \leftarrow B[i]$; $i \leftarrow i + 1$

else $A[k] \leftarrow C[j]$; $j \leftarrow j + 1$

$k \leftarrow k + 1$

if $i = p$

 copy $C[j..q-1]$ to $A[k..p+q-1]$

else copy $B[i..p-1]$ to $A[k..p+q-1]$

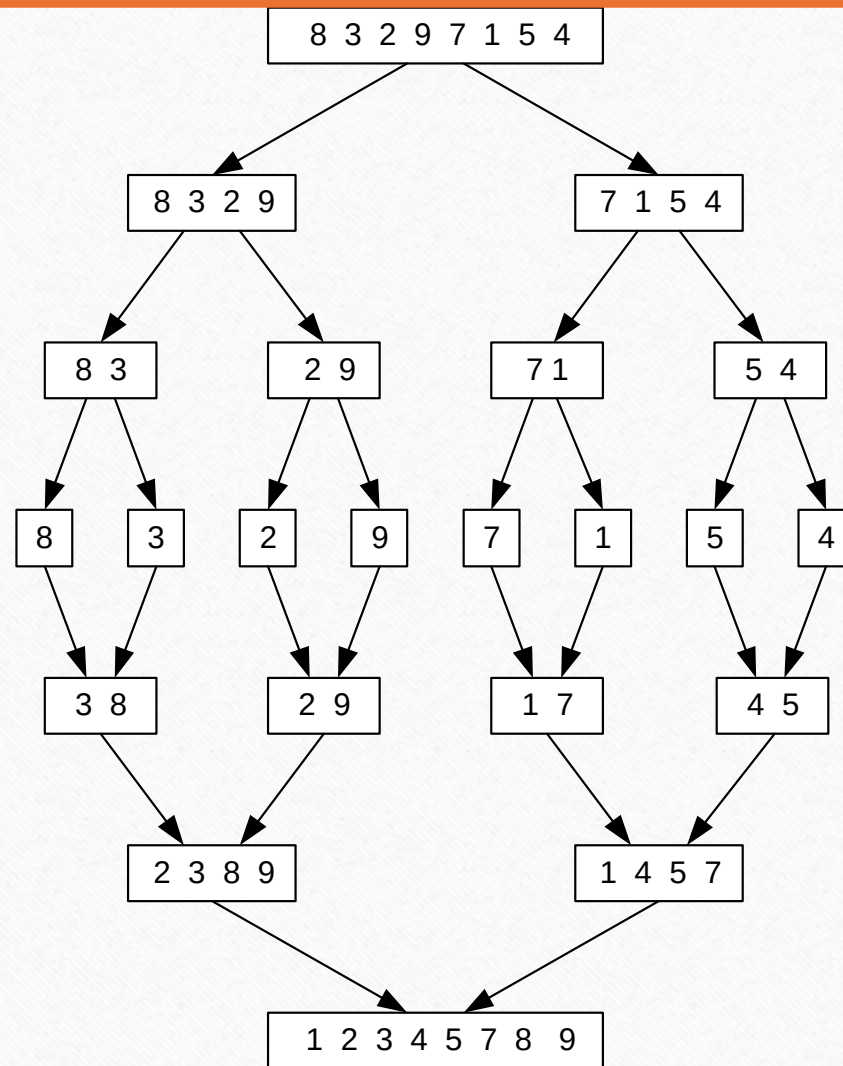
Time complexity: $\Theta(p + q) = \Theta(n)$
comparisons

1. Initialize pointers i , j , and k to 0.
2. Compare $B[i]$ with $C[j]$.
3. If $B[i]$ is greater than $C[j]$, set $A[k]$ to $C[j]$ and increment k and j .
4. If $B[i]$ is less than $C[j]$, set $A[k]$ to $B[i]$ and increment k and i .
5. Repeat steps 2-4 until one of the arrays (B or C) is exhausted.
6. If elements remain in B , copy them into A .
7. If elements remain in C , copy them into A .

Merge

- $B=[2, 4, 15, 20]$ $C=[1, 16, 100, 120]$ merge B & C in A
- $i=0$ $j=0$ $k=0$
- $B[i]=2 > C[j]=1$ therefore $A[0]=1$ $k=1, j=1$
- $B[i]=2 < C[j]=16$ therefore $A[1]=2$ $k=2, i=1$
- $B[i]=4 < C[j]=16$ $A[2]=4$ $i=2, k=3$
- $B[i]=15 < C[j]=16$ $A[3]=15$ $i=3, k=4$
- $B[i]=20 > C[j]=16$ $A[4]=16$ $k=4, j=2$
- $B[i]=20 < C[j]=100$ $A[5]=20$ $k=5, i=4$
- $i>3$ therefore copy remaining element of C in A
- $A[6]=100$ $A[7]=120$
- $A=[1, 2, 4, 15, 16, 20, 100, 120]$

Merge sort Example



- Apply Merge sort Technique and Draw the recursion Tree for the following Sequence:
- 8,3,2,9,7,1,5,4

Analysis of Mergesort

Let $T(n)$ denote time complexity of sorting n elements using merge sort then :

$$T(n) = 2T(n/2) + cn, T(1) = 0 \text{ for some constant } c$$

Analysis of Merge Sort

$T(n) = 0$ (if there is only one element)

$T(n) = 2T(n/2) + n$ (Suppose if there are > 1 element)

↓
minimum no of comparison.

$$= 2[2T(n/2/2) + n/2] + n \rightarrow 2^1 T(n/2^1) + 1 \cdot n$$

$$= 4T(n/4) + n + n$$

$$= 4T(n/4) + 2n$$

$$\Rightarrow 2^2 T(n/2^2) + 2n$$

$$= 4[2T(n/4/2) + n/4] + 2n$$

$$= 8T(n/8) + n + 2n$$

$$= 8T(n/8) + 3n \Rightarrow 2^3 T(n/2^3) + 3n$$

⋮
⋮
⋮

$$T(n) \Rightarrow 2^k T(n/2^k) + kn \quad \text{--- (i)}$$

$$\text{Let } 2^k = n$$

$$\text{apply log, } \log 2^k = \log n$$

$$k \log 2 = \log n$$

$$k \times 1 = \log n$$

$$\underline{\underline{k = \log n}}$$

$$(\log_2 2 = 1)$$

∴ Substitute in (1)

$$T(n) = n T(n/n) + \log n \cdot n$$

$$= n T(1) + n \log(n)$$

$$= n \times 1 + n \log n$$

$$= 0 + n \log n$$

$$= \therefore \underline{O(n \log n)}$$

Analysis of Merge sort

For simplification purpose assume $n = 2^k$ for some positive constant k

$T(n) = 2T(n/2) + Cn$, where c is a constant

$T(n) = 2[2T(n/2^2) + Cn/2] + Cn = 2^2 T(n/2^2) + 2Cn$

$T(n) = 2^2[2T(n/2^3) + Cn/2^2] + 2Cn$

$T(n) = 2^3 T(n/2^3) + 3Cn$

.....

.....

.....

$T(n) = 2^k T(n/2^k) + kCn$

Therefore, $T(n) = 2^k T(1) + kCn = kCn$ (since $T(1)=0$)

Which implies $T(n) = O(kn) = O(n \log n)$

[since $n = 2^k$, $k = \log n$]

Analysis of Merge sort (Using Masters Theorem)

$$T(n) = aT(n/b) + f(n) \quad \text{where } f(n) = \Theta(n^d), \\ d \geq 0$$

Master Theorem: If $a < b^d$, $T(n) = \Theta(n^d)$
If $a = b^d$, $T(n) = \Theta(n^d \log n)$
If $a > b^d$, $T(n) = \Theta(n^{\log_b a})$

$$T(n) = 2T(n/2) + cn, \quad T(1) = 0 \quad \text{for some constant } c$$

Comparing the Recurrence Relation with the Masters Theorem we get
 $a = 2, b = 2, d = 1$
Using Masters Theorem
 $a = b^d$

$$\text{Therefore } T(n) = \Theta(n^d \log n)$$

$$\Rightarrow T(n) = \Theta(n \log n)$$



Quick Sort

Quick Sort

It is also known as "partition-exchange sort"

Quick sort is a wellknown sorting algorithm, based on divide & conquer approach. The steps are:

- ▶ Pick an element called pivot from the list
- ▶ Reorder the list so that all elements which are less than the pivot come before the pivot and all elements greater than pivot come after it. After this partitioning, the pivot is in its final position. This is called the partition operation
- ▶ Recursively sort the sub-list of lesser elements and sub-list of greater elements.

Quick Sort

- Small instance has $n \leq 1$. Every small instance is a sorted instance.
- To sort a large instance, select a **pivot** element from out of the n elements.
- Partition the n elements into 3 groups **left**, **middle** and **right**.
- The **middle** group contains only the **pivot** element.
- All elements in the **left** group are $\leq$ **pivot**.
- All elements in the **right** group are $\geq$ **pivot**.
- Sort **left** and **right** groups recursively.
- Answer is sorted **left** group, followed by **middle** group followed by sorted **right** group.

Example

6	2	8	5	11	10	4	1	9	7	3
---	---	---	---	----	----	---	---	---	---	---

Use 6 as the pivot.

								10	11	
--	--	--	--	--	--	--	--	----	----	--

Sort left and right groups recursively.

Choice Of Pivot

- Pivot is **leftmost** element in list that is to be sorted.
 - When sorting $a[6:20]$, use $a[6]$ as the pivot.
- **Randomly** select one of the elements to be sorted as the pivot.
 - When sorting $a[6:20]$, generate a random number r in the range $[6, 20]$. Use $a[r]$ as the pivot.

Choice Of Pivot

- **Median-of-Three rule.** From the leftmost, middle, and rightmost elements of the list to be sorted, select the one with median key as the pivot.
 - When sorting $a[6:20]$, examine $a[6]$, $a[13]$ $((6+20)/2)$, and $a[20]$. Select the element with median (i.e., middle) key.

Quick Sort Algorithm

```
ALGORITHM Quicksort (A[ l    r  ])
//Input: A sub-array A[l..r] of A[0..n-1],
defined by its left and right indices l and r
//Output: The sub-array A[l..r],
          sorted in ascending order
if l < r
s =Partition (A[l..r]) // s is a split position
Quicksort(A[l..s-1])
Quicksort(A[s+1..r])
```

Partitions a sub-array by using its first element as a pivot

ALGORITHM Partition ($A[l \dots r]$)

//Input: A sub-array $A[l \dots r]$ of $A[0 \dots n-1]$,

defined by its left and right indices l and r

//Output: A partition of $A[l \dots r]$, with split posi

$p = A[l]$, $i = l$, $j = r + 1$;

Repeat

repeat $i = i + 1$ until $A[i] \geq p$

repeat $j = j - 1$ until $A[j] < p$

if ($i < j$)

swap($A[i]$, $A[j]$)

until $i \geq j$

swap($A[l]$, $A[j]$)

return j

Partitions a sub-array by using its first element as a pivot

ALGORITHM Partition (A[1 ..r])

//Input: A sub-array A[1..r] of A [0..n-1],

**defined by its left and right indices 1 and r //Output: A partition of
A[1..r], with split pos:**

p=A[1], i=1,j= r + 1;

Repeat

repeat i=i+1 until A [i] <= p

repeat j=j-1until A [j] > p

if (i < j)

swap (A[i], a[j])

until i >= j

swap (A [1], A[j])

return j

- Otherwise,
- In the following iterations, the size of the subarray is reduced using the result of the previous comparison.
- Initial length of array $= n$
- Iteration 1 - Length of array $= n/2$
- Iteration 2 - Length of array $= (n/2)/2 = n/2^2$
- Iteration k - Length of array $= n/2^k$
the length of the array is halved.

After k iterations, the size of the array becomes 1 (narrowed down to the first element or last element only).

$$\text{Length of array} = n/2^k = 1$$

$$\Rightarrow n = 2^k$$

Applying log function on both sides:

$$\Rightarrow \log_2(n) = \log_2(2^k)$$

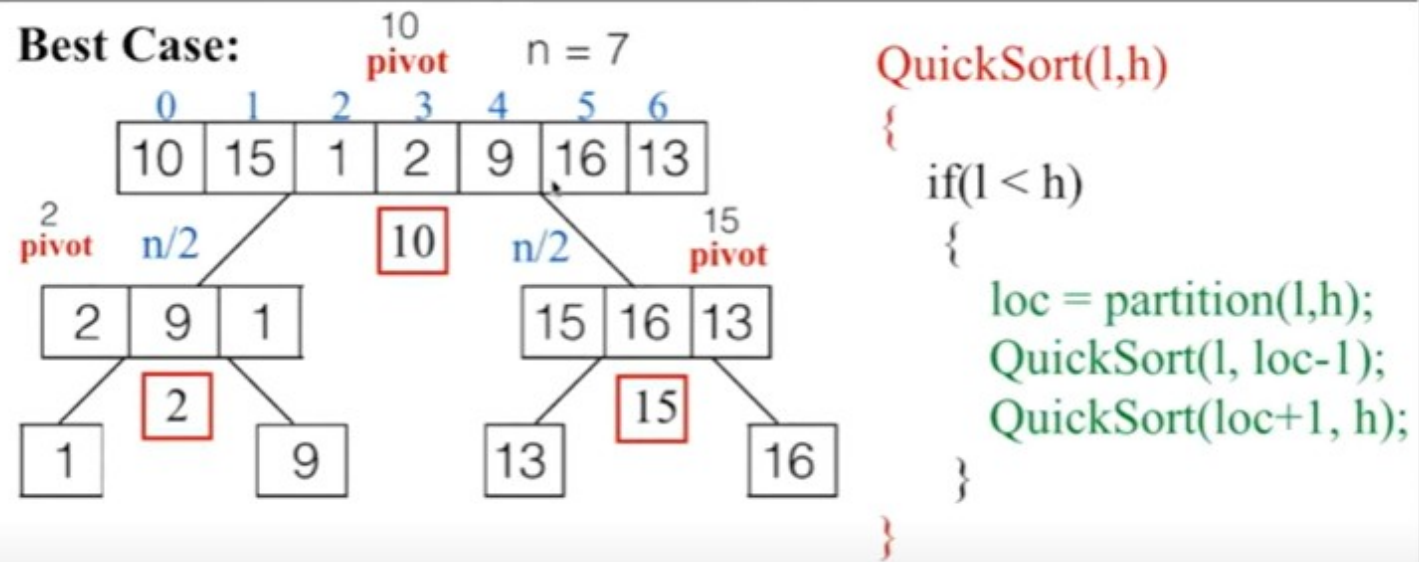
$$\Rightarrow \log_2(n) = k * \log_2 2 = k$$

$$\Rightarrow k = \log_2(n)$$

Best case: when partition happens in the middle of the array each time.

- Recursion Tree = No of levels * Time taken for one Level
- $= \log n * n = O(n \log n)$ Here ($\log n$ is same as merge sort)

Partition tree

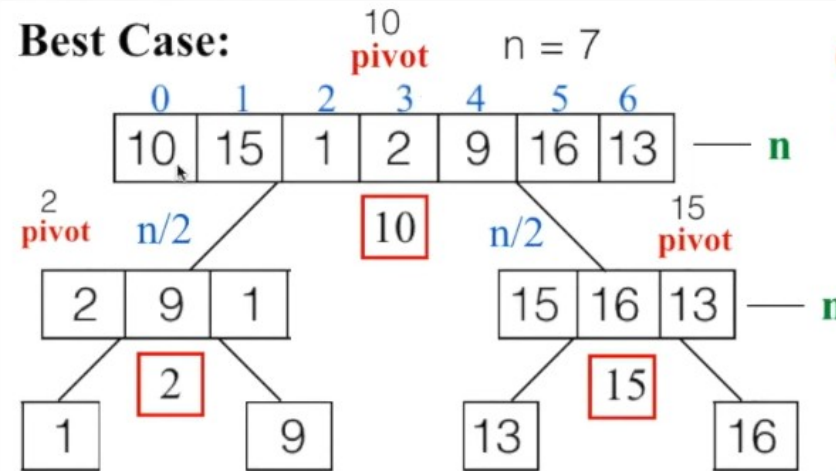


Equal number of elements in the



Analysis of Quick Sort

Best Case:



QuickSort(l,h)

{
if(l < h)

{

loc = partition(l,h);
QuickSort(l, loc-1);
QuickSort(loc+1, h);

}

}

Time complexity of

entire recursion tree : No.of levels * Time taken for one level

$$\frac{n}{2^k} = 1$$

$$\log_2 n = k \log_2 2$$

$$\log_2 n * n$$

Best Case: $O(n * \log n)$

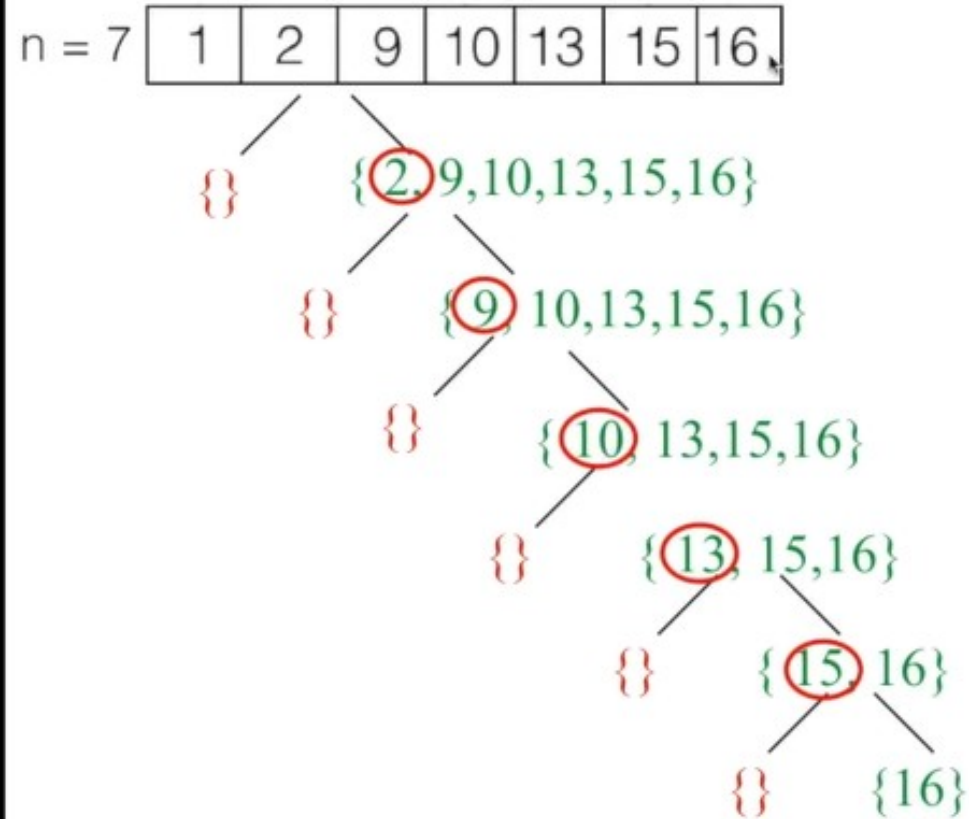
$$n = 2^k$$

$$k = \log_2 n$$



Analysis of Quick Sort

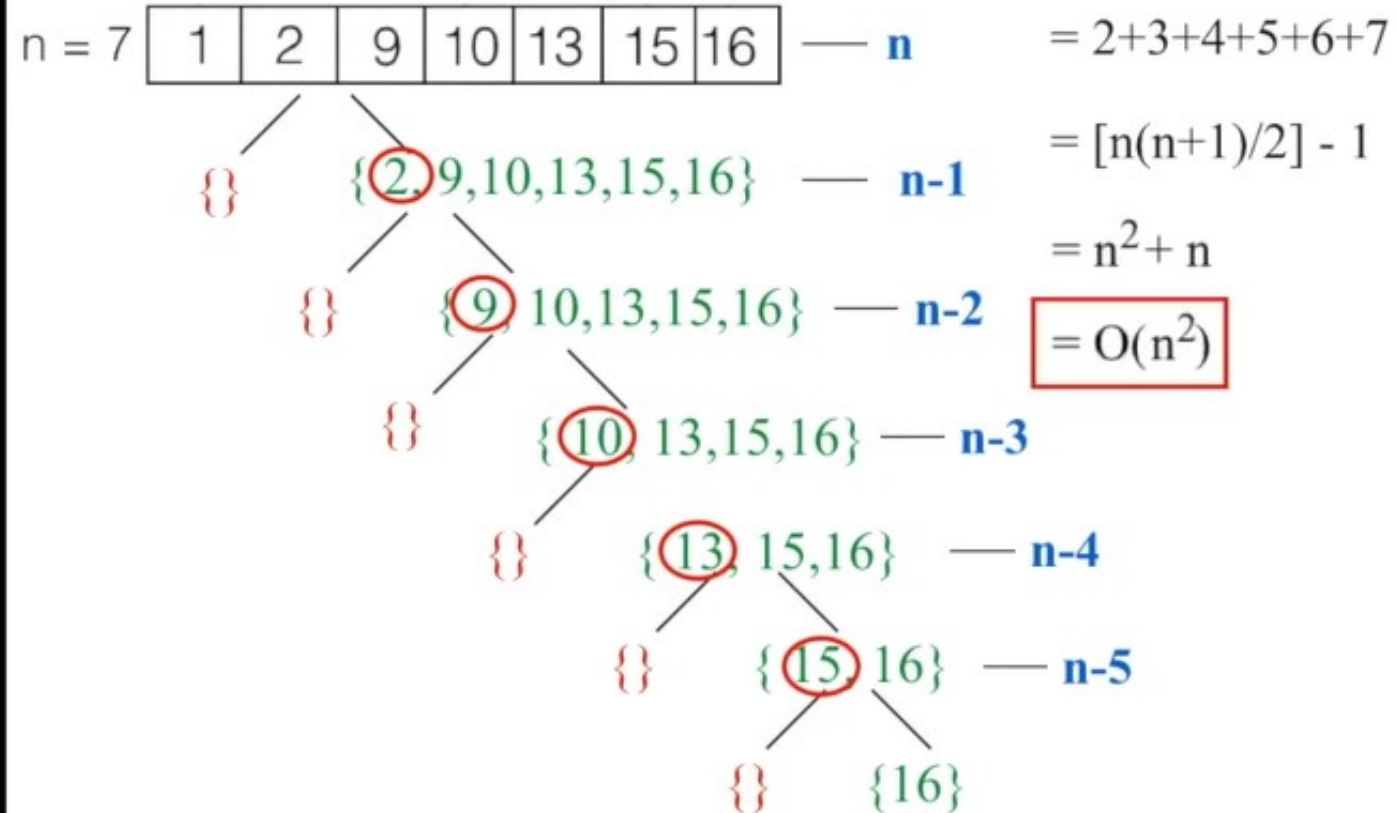
Worst Case: Array is already Sorted.





Analysis of Quick Sort

Worst Case: Array is already Sorted.



Analysis of Quicksort

Best case: split in the middle (same as merge sort)— $O(n \log n)$

Worst case: sorted array! — $O(n^2)$

$$T(n) = T(n-1) + \Theta(n) , \quad T(1) = 0$$

$$T(n) = T(n-1) + cn = T(n-2) + c(n-1) + cn$$

.....

.....

$$T(n) = T(1) + c(n + (n-1) + \dots + 2) = O(n^2)$$

Average case: random arrays — $\Theta(n \log n)$?

Analysis of Quick Sort

Best Case (Using Masters Theorem)

$$T(n) = aT(n/b) + f(n) \quad \text{where } f(n) = \Theta(n^d), \\ d \geq 0$$

Master Theorem: If $a < b^d$, $T(n) = \Theta(n^d)$
If $a = b^d$, $T(n) = \Theta(n^d \log n)$
If $a > b^d$, $T(n) = \Theta(n^{\log_b a})$

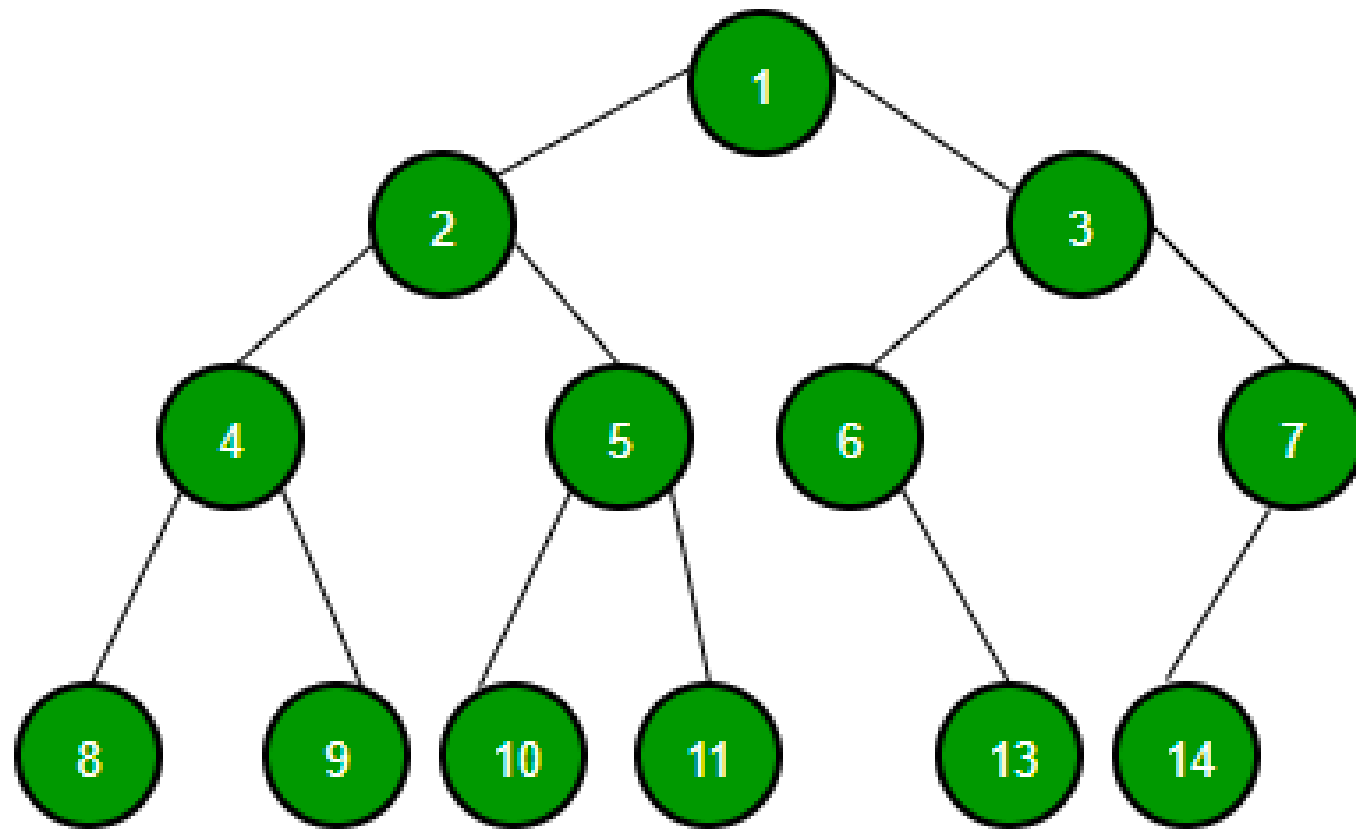
$$T(n) = 2T(n/2) + Cn$$

Comparing the Recurrence Relation with the Masters Theorem we get
 $a = 2, b = 2, d = 1$
Using Masters Theorem
 $a = b^d$

Therefore $T(n) = \Theta(n^d \log n)$

$$\Rightarrow T(n) = \Theta(n \log n)$$

Binary Tree Traversals and Related Properties



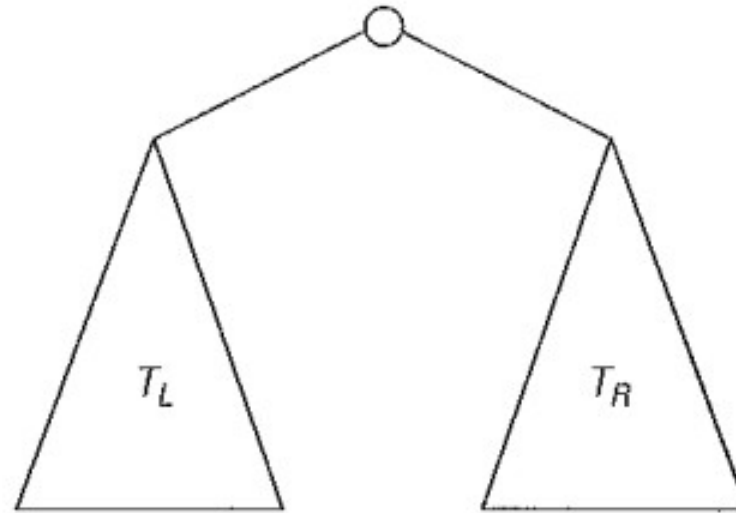


FIGURE 4.4 Standard representation of a binary tree

- A binary tree T is defined as a finite set of nodes that is either empty or consists of a root and two disjoint binary trees T_L and T_R called, respectively, the left and right subtree of the root.

ALGORITHM *Height*(T)

//Computes recursively the height of a binary tree

//Input: A binary tree T

//Output: The height of T

if $T = \emptyset$ **return** -1

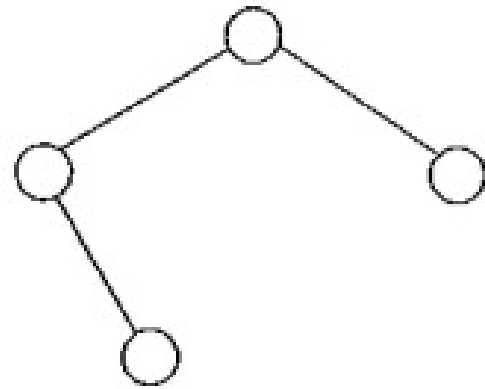
else return $\max\{Height(T_L), Height(T_R)\} + 1$

We measure the problem's instance size by the number of nodes $n(T)$ in a given binary tree T .

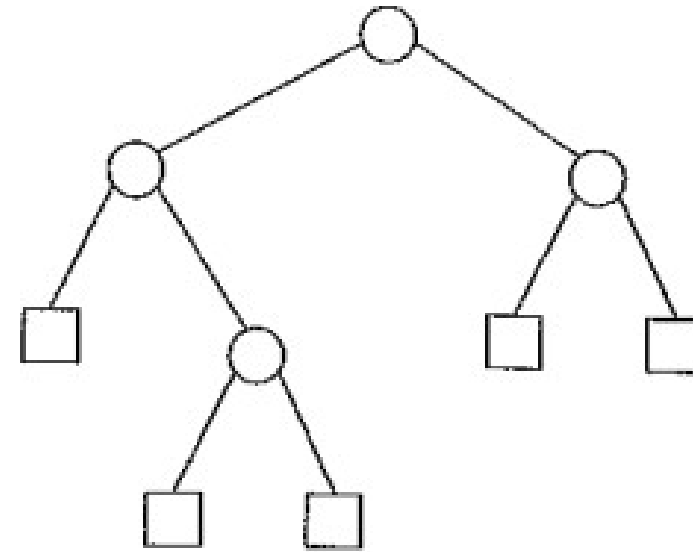
Obviously, the number of comparisons made to compute the maximum of two numbers and the number of additions $A(n(T))$ made by the algorithm are the same.

We have the following recurrence relation for $A(n(T))$:

$$\mathbf{A(n(T)) = A(n(TL)) + A(n(TR)) + 1 \text{ for } n(T) > 0, A(0) = 0.}$$



(a)

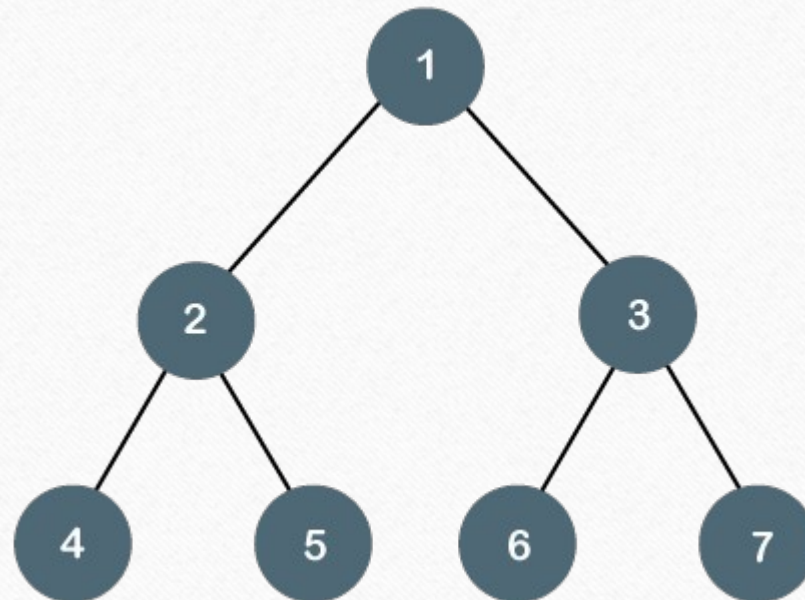


(b)

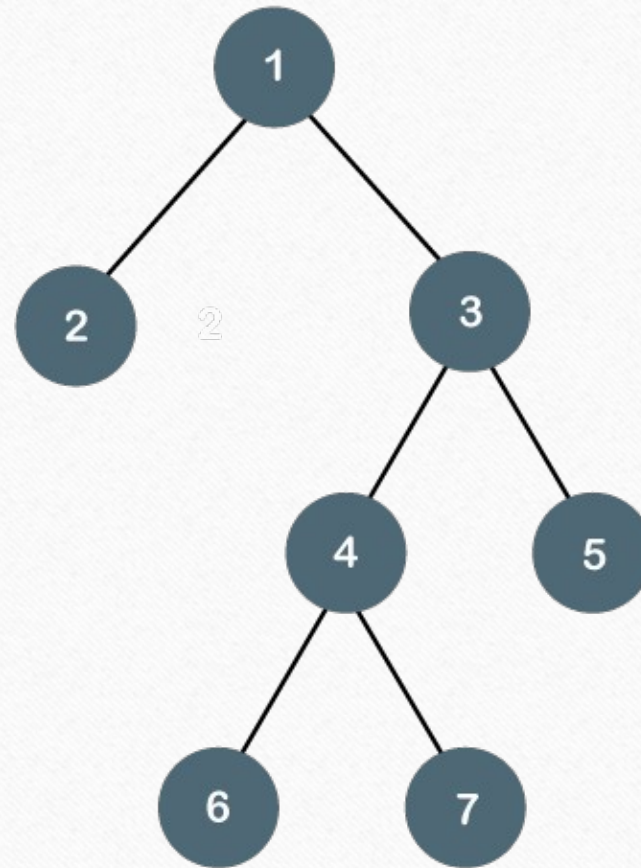
FIGURE 4.5 (a) Binary tree. (b) Its extension. Internal nodes are shown as circles; external nodes are shown as squares.

- It is easy to see that the height algorithm makes exactly one addition for every internal node of the extended tree, and it makes one comparison to check whether the tree is empty for every internal and external node.
- Thus, to ascertain the algorithm's efficiency, we need to know how many external nodes an extended binary tree with n internal nodes can have. Checking Figure 4.5 and a few similar examples, it is easy to hypothesize that the number of external nodes x is always one more than the number

Perfect Binary Tree

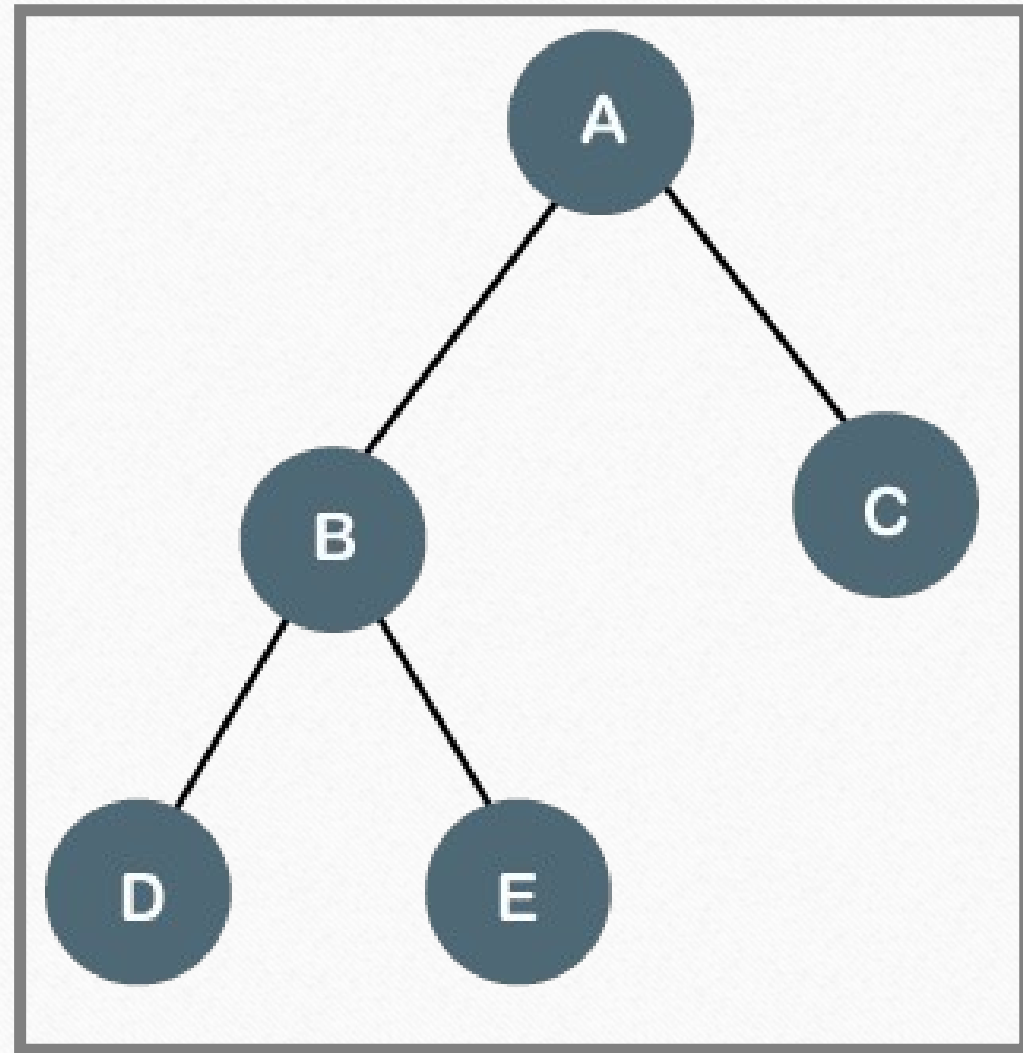


The below tree is not a perfect binary tree because all the leaf nodes are not at the same level.



Full/ proper/ strict Binary tree

- In the above tree, we can observe that each node is either containing zero or two children; therefore, it is a Full Binary tree.



In the preorder traversal, the root is visited before the left and right subtrees are visited (in that order).

In the inorder traversal, the root is visited after visiting its left subtree but before visiting the right subtree.

In the postorder traversal, the root is visited after visiting the left and right subtrees (in that order).

Traversal of Binary Tree

- Traversal methods

- **Inorder traversal: LCR**

- Visiting a left subtree, a root node, and a right subtree

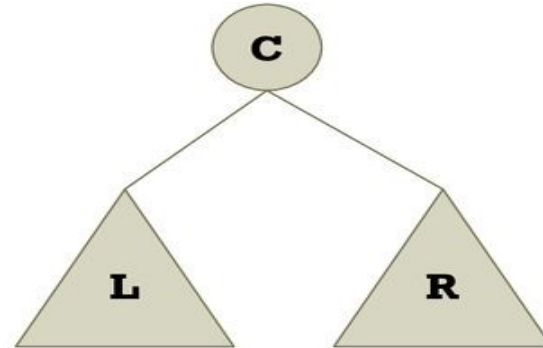
- **Preorder traversal: CLR**

- Visiting the root node before subtrees

- **Postorder traversal: LRC**

- Visiting subtrees before visiting the root node

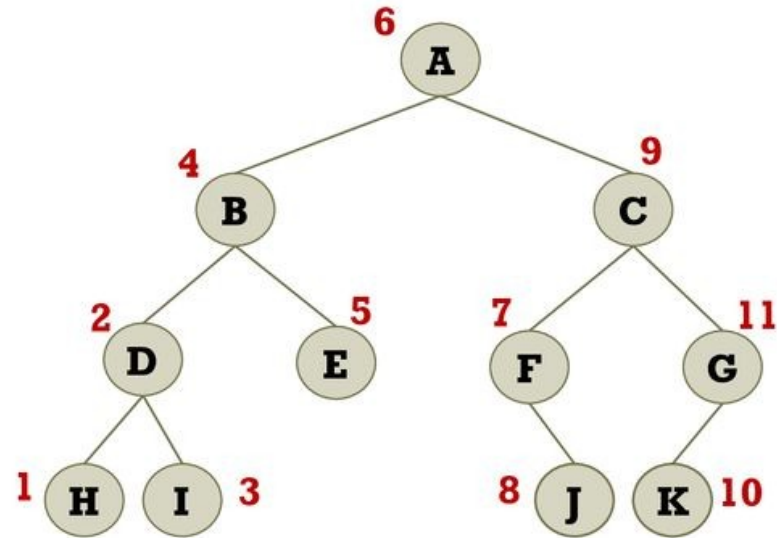
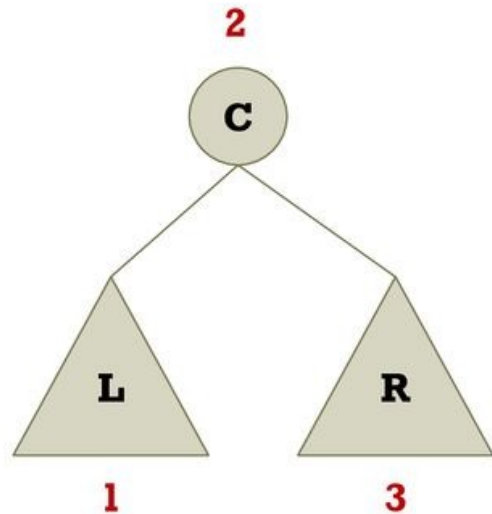
- **Level order traversal**



Inorder Traversal (LCR)

■ Algorithm: LCR

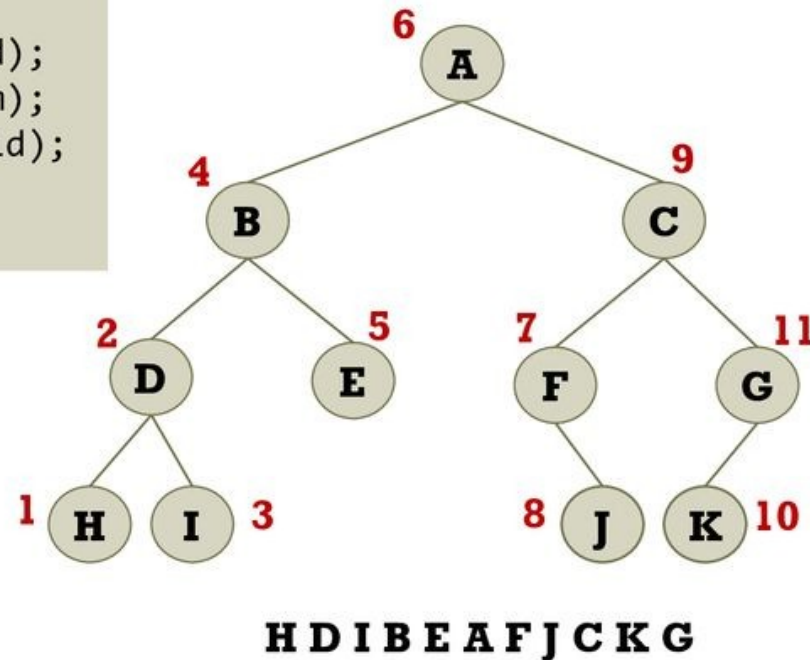
- Step 1: Visiting a left subtree
- Step 2: Visiting the root node
- Step 3: Visiting a right subtree



Inorder Traversal (LCR)

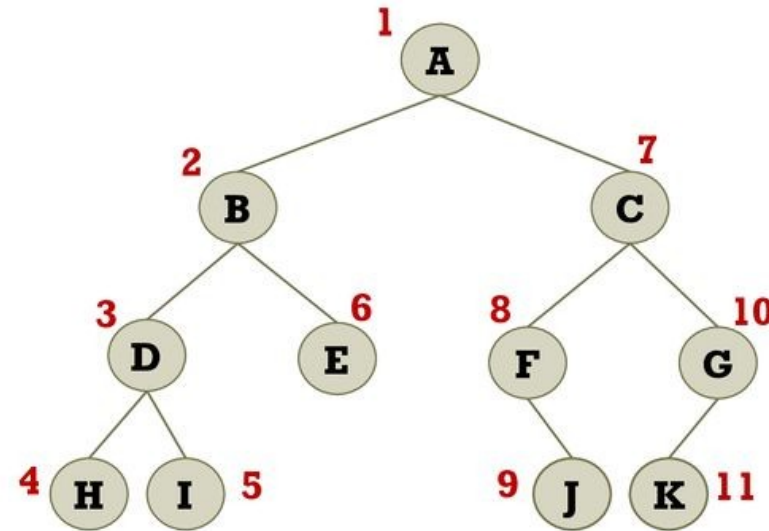
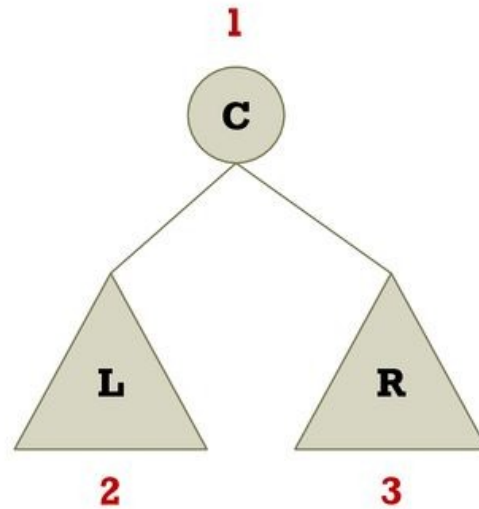
■ Algorithm

```
void Inorder(BTreeNode* root)
{
    if (root != NULL)
    {
        Inorder(root->left_child);
        printf("%d ", root->item);
        Inorder(root->right_child);
    }
}
```



Preorder Traversal (CLR)

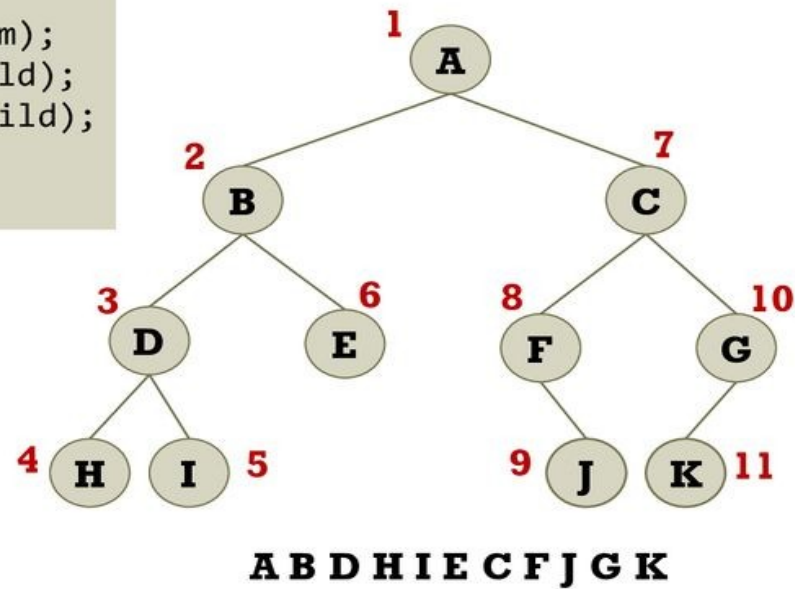
- Algorithm: CLR
 - Step 1: Visiting the root node
 - Step 2: Visiting a left subtree
 - Step 3: Visiting a right subtree



Preorder Traversal (CLR)

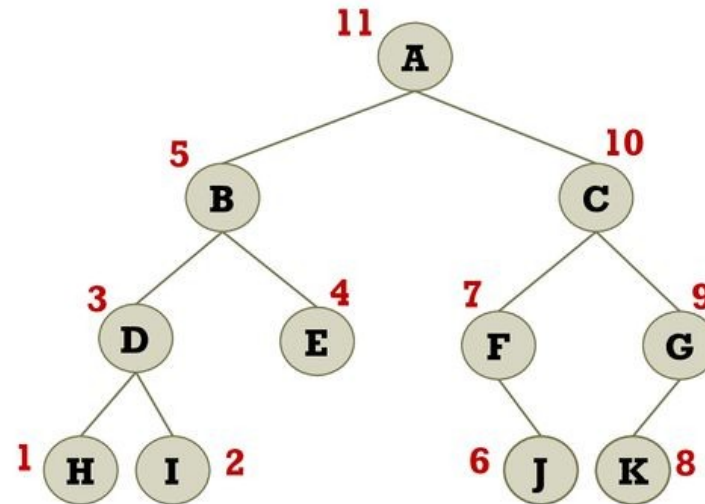
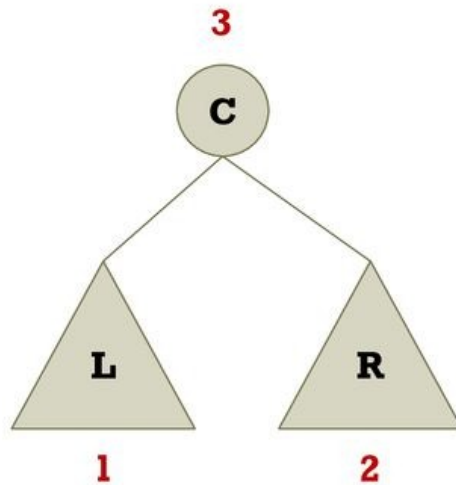
■ Algorithm: CLR

```
void Preorder(BTreeNode* root)
{
    if (root != NULL)
    {
        printf("%d ", root->item);
        Preorder(root->left_child);
        Preorder(root->right_child);
    }
}
```



Postorder Traversal (LRC)

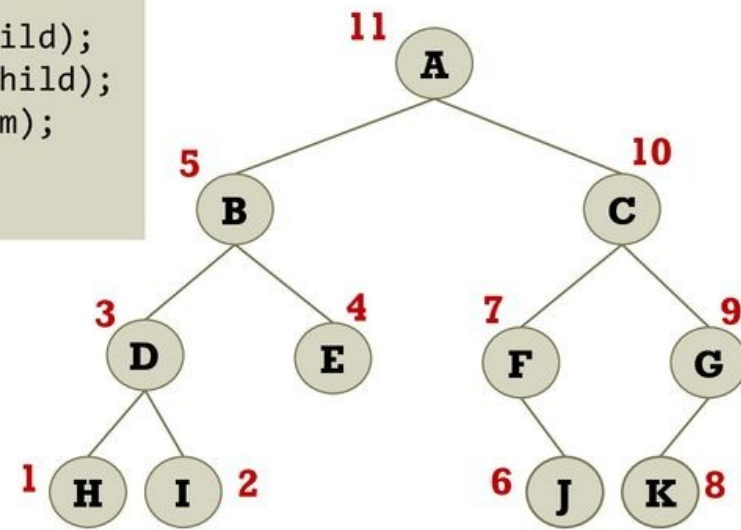
- Algorithm: LRC
 - Step 1: Visiting a left subtree
 - Step 2: Visiting a right subtree
 - Step 3: Visiting the root node



Postorder Traversal (LRC)

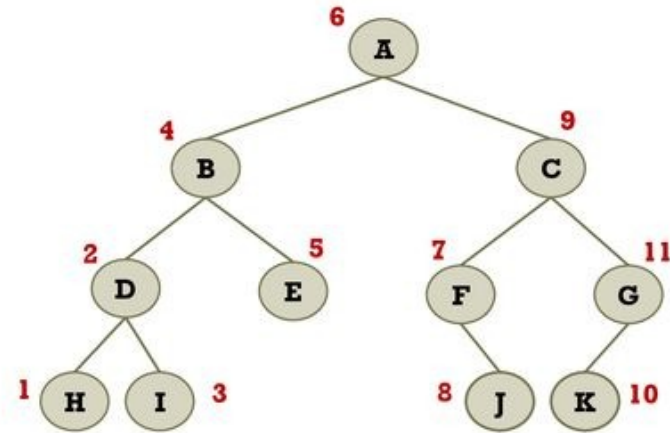
■ Algorithm: LRC

```
void Postorder(BTreeNode* root)
{
    if (root != NULL)
    {
        Postorder(root->left_child);
        Postorder(root->right_child);
        printf("%d ", root->item);
    }
}
```

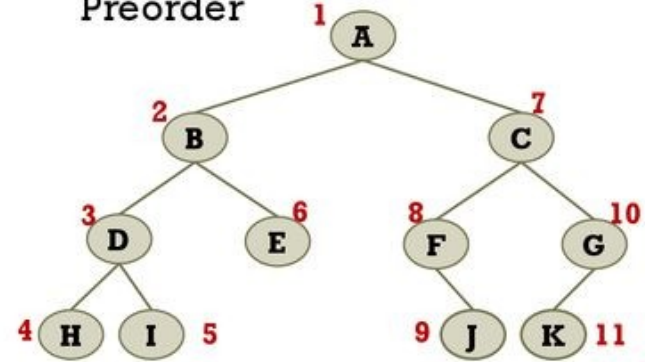


H I D E B J F K G C A

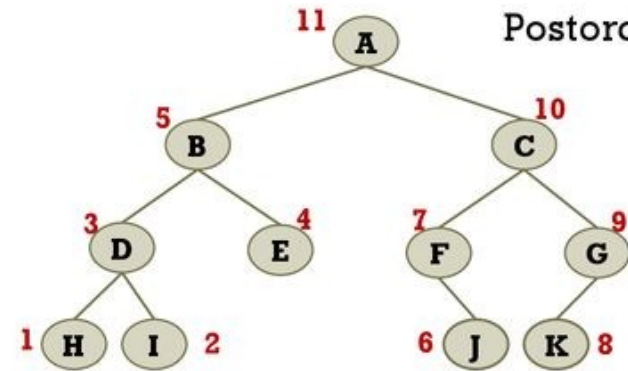
Inorder



Preorder



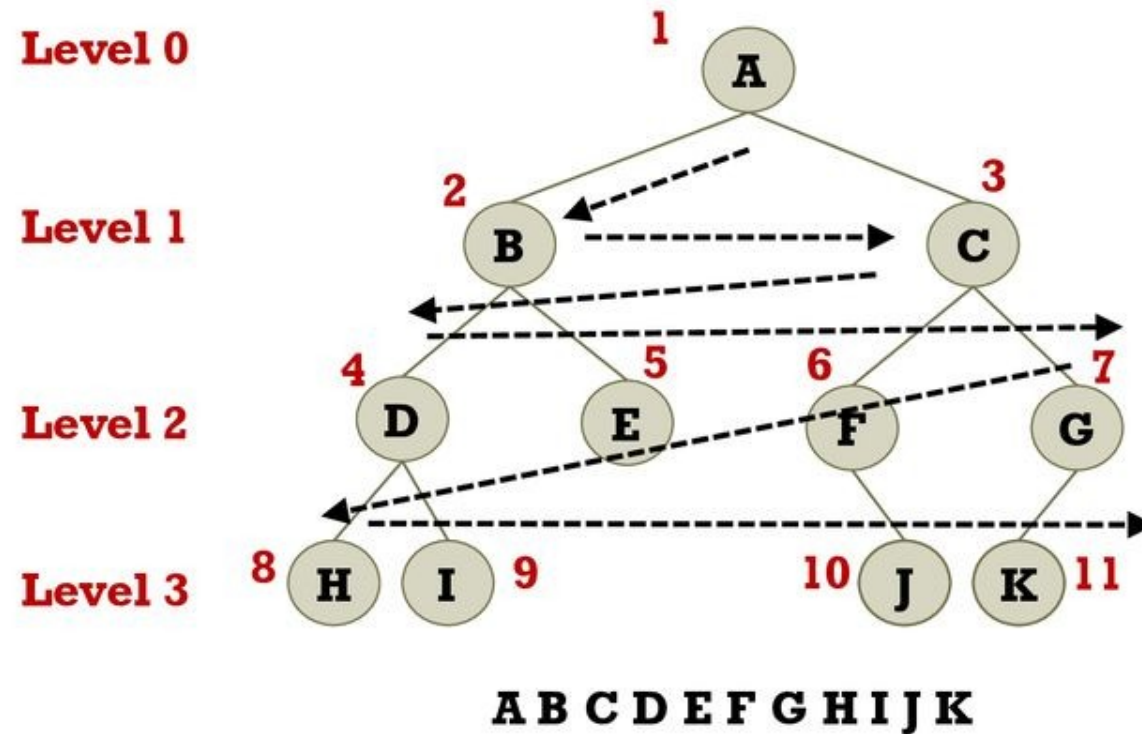
Postorder



Level Order Traversal

■ Algorithm

- For each level, visit nodes from the left to right direction.

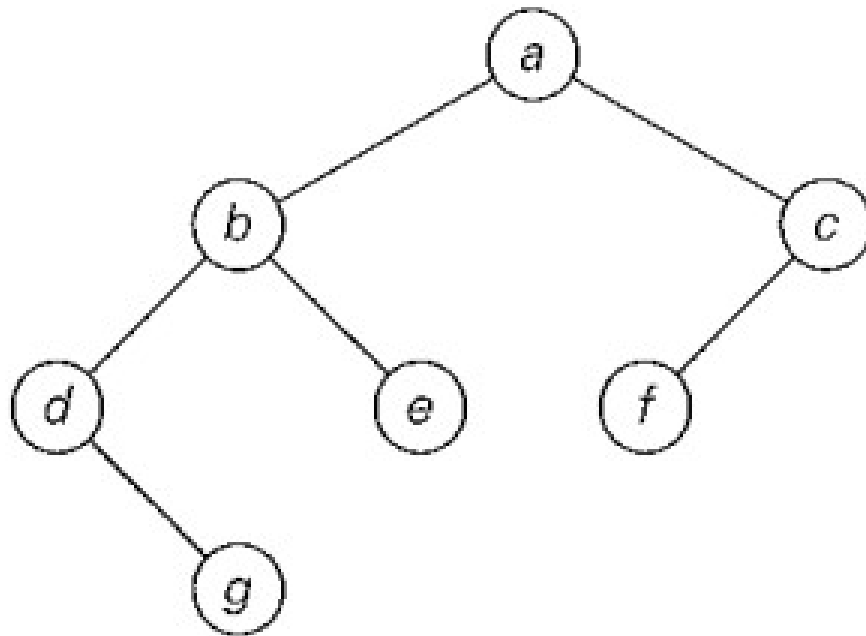


Level Order Traversal

```
void Levelorder(BTreeNode* root)
{
    Queue queue;
    if (root == NULL) return;

    InitQueue(&queue);
    EnQueue(&queue, root);
    while (!IsEmpty(&queue))
    {
        root = Peek(&queue);
        DeQueue(&queue);

        printf("%d ", root->item);
        if (root->left_child != NULL)
            EnQueue(&queue, root->left_child);
        if (root->right_child != NULL)
            EnQueue(&queue, root->right_child);
    }
}
```



Preorder: a, b, d, g, e, c, f

Inorder: d, g, b, e, a, f, c

Postorder: g, d, e, b, f, c, a

FIGURE 4.6 Binary tree and its traversals



Matrix Multiplication

Conventional Matrix Multiplication

Example 18.1 (matrix multiplication) The product of two $n \times n$ matrices A and B is a third $n \times n$ matrix C where $C(i, j)$ is given by

$$C(i, j) = \sum_{k=1}^n A(i, k) * B(k, j), \quad 1 \leq i \leq n, \quad 1 \leq j \leq n \quad (18.1)$$

```
for (int i=0; i< n ; i++)  
for (int j=0; j<n; j++)  
{  
    c[i][j]=0;  
    for (int k=0; i<n; k++)  
        c[i][j]+= A[i][k]*B[k][j];  
}
```

$O(n^3)$

- Suppose we have a 2×2 matrix A, which has 2 rows and 2 columns:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}$$

- Suppose we also have a 2×2 matrix B, which has 2 rows and 2 columns:

$$B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

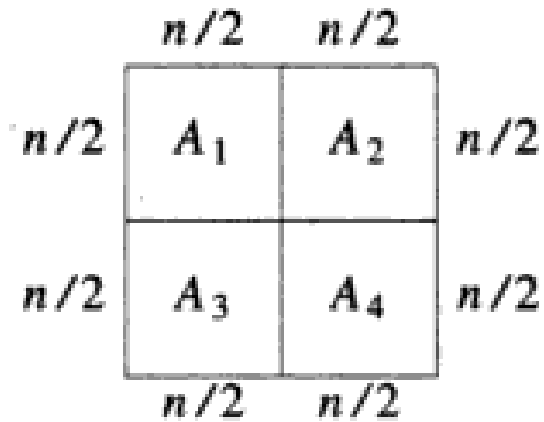
- To multiply matrix A by matrix B we use the following formula:

$$A \times B = \begin{bmatrix} A_{11} * B_{11} + A_{12} * B_{21} & A_{11} * B_{12} + A_{12} * B_{22} \\ A_{21} * B_{11} + A_{22} * B_{21} & A_{21} * B_{12} + A_{22} * B_{22} \end{bmatrix}$$

This results in a 2×2 matrix.

- We need to use 3 for loop.
- Directly use the formula .So each statement will take 1 unit of time so Total time is 4 Unit.That is constant.
- So this is is defined as the small Problem.
- If you want to reduce the size and say 1x1 matrix,then also it is small problem.
- Now if >2 then????

Strassen's Matrix Multiplication

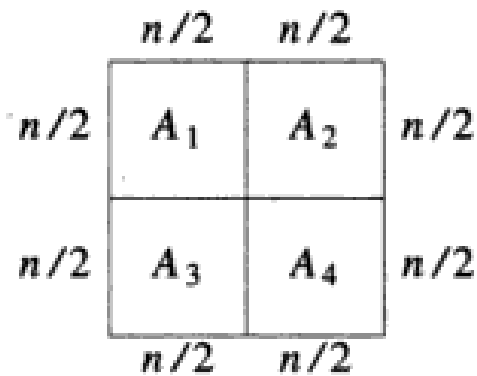


(a) Dividing A into four

$$\begin{bmatrix} A_1 & A_2 \\ A_3 & A_4 \end{bmatrix} * \begin{bmatrix} B_1 & B_2 \\ B_3 & B_4 \end{bmatrix} = \begin{bmatrix} C_1 & C_2 \\ C_3 & C_4 \end{bmatrix}$$

(b) $A * B = C$

Matrix Multiplication : Divide and Conquer



(a) Dividing A into four

$$\begin{bmatrix} A_1 & A_2 \\ A_3 & A_4 \end{bmatrix} * \begin{bmatrix} B_1 & B_2 \\ B_3 & B_4 \end{bmatrix} = \begin{bmatrix} C_1 & C_2 \\ C_3 & C_4 \end{bmatrix}$$

(b) $A * B = C$

$$C_1 = A_1 B_1 + A_2 B_3$$

$$C_2 = A_1 B_2 + A_2 B_4$$

$$C_3 = A_3 B_1 + A_4 B_3$$

$$C_4 = A_3 B_2 + A_4 B_4$$

$$\mathbf{T(n) = 8 T(n/2) + cn^2 \text{ for } n \geq 2 \text{ and } T(1)=d}$$

Solve ! ?

Algo MM(A,B,n)

{

if(n<=2){ c=apply 4 formula;}

Else

{

Mid=(n/2)

MM(A1,B1,n/2)+MM(A2,B3,n/2)

MM(A1,B2,n/2)+MM(A2,B4,n/2)

MM(A3,B1,n/2)+MM(A4,B3,n/2)

MM(A3,B2,n/2)+MM(A4,B4,n/2)}}}

- $T(n) = \begin{cases} 1 & \text{if } n \leq 2 \\ 8T(n/2) + n^2 & \text{if } n > 2 \end{cases}$

- **Now Apply Masters Theorem,**
- **$a=8$ $b=2$ $f(n)=n^2$ $d=2$**
- **So $O(n^3)$**

- $T(n) = \begin{cases} 1 & \text{if } n \leq 2 \\ 8T(n/2) + n^2 & \text{if } n > 2 \end{cases}$

- Now Apply Masters Theorem,
- $a=8$ $b=2$ $f(n)=n^2$ $d=2$
- So $O(n^3)$

- $T(n) = 8 T(n/2) + cn^2 = 8 [8T(n/2^2) + cn^2/4] + cn^2$
- $= 8^2T(n/2^2) + cn^2 [1 + 2]$
- $= 8^2[8T(n/2^3) + cn^2/16] + cn^2 [1 + 2 + 4]$
- $= 8^3T(n/2^3) + cn^2 [1 + 2 + 2^2]$
-
-
- $= 8^kT(n/2^k) + cn^2 [1 + 2 + 2^2 + \dots + 2^{k-1}] =$
 $O(n^3)$ (Since $n = 2^k$)

Analysis of Matrix Multiplication (Using Masters Theorem)

$$T(n) = aT(n/b) + f(n) \quad \text{where } f(n) = \Theta(n^d), \\ d \geq 0$$

Master Theorem: If $a < b^d$, $T(n) = \Theta(n^d)$
 If $a = b^d$, $T(n) = \Theta(n^d \log n)$
 If $a > b^d$, $T(n) = \Theta(n^{\log_b a})$

$$T(n) = 8 T(n/2) + cn^2$$

Comparing the Recurrence Relation with the Masters Theorem we get
 $a = 8, b = 2, d = 2$

Using Masters Theorem
 $a > b^d$

$$\text{Therefore } T(n) = \Theta(n^{\log_b a}) \\ \Rightarrow T(n) = \Theta(n^{\log_2 8}) = \Theta(n^3)$$

- $T(n) = 8 T(n/2) + cn^2 = 8 [8T(n/2^2) + cn^2/4] + cn^2$
- $= 8^2 T(n/2^2) + cn^2 [1 + 2]$
- $= 8^2 [8T(n/2^3) + cn^2/16] + cn^2 [1 + 2 + 4]$
- $= 8^3 T(n/2^3) + cn^2 [1 + 2 + 2^2]$
-
-
- $= 8^k T(n/2^k) + cn^2 [1 + 2 + 2^2 + \dots + 2^{k-1}] = O(n^3)$ (Since $n = 2^k$)

Formulas for Strassen's Algorithm

$$D = A_1(B_2 - B_4)$$

$$E = A_4(B_3 - B_1)$$

$$F = (A_3 + A_4)B_1$$

$$G = (A_1 + A_2)B_4$$

$$H = (A_3 - A_1)(B_1 + B_2)$$

$$I = (A_2 - A_4)(B_3 + B_4)$$

$$J = (A_1 + A_4)(B_1 + B_4)$$

$$C_1 = E + I + J - G$$

$$C_2 = D + G$$

$$C_3 = E + F$$

$$C_4 = D + H + J - F$$

$$T(n) = 7 T(n/2) + cn^2 \quad T(2) = d, \text{ for some constant } d$$

Analysis of Strassen's Algorithm

$$T(n) = 7 T(n/2) + cn^2 \quad T(1) = d, \text{ for some constant } d$$

$$\begin{aligned} T(n) &= 7 T(n/2) + cn^2 = 7 [7 T(n/2^2) + c(n/2)^2] + cn^2 = \\ &= 7^2 T(n/2^2) + cn^2 [1 + 7/4] \\ &= 7^3 T(n/2^3) + cn^2 [1 + 7/4 + (7/4)^2] \end{aligned}$$

.....

.....

$$\begin{aligned} &= 7^k T(n/2^k) + cn^2 [1 + 7/4 + (7/4)^2 + \\ &\dots (7/4)^{k-1}] \end{aligned}$$

$$\begin{aligned} &= 7^k T(n/2^k) + cn^2 (7/4)^k / [7/4 - 1] = \\ O(7^k) & \\ &= O(7^{\log n}) = O(n^{\log 7}) = O(n^{2.81}) \end{aligned}$$

Analysis of Strassen's Algorithm (Using Masters Theorem)

$$T(n) = aT(n/b) + f(n) \quad \text{where } f(n) = \Theta(n^d), \\ d \geq 0$$

Master Theorem: If $a < b^d$, $T(n) = \Theta(n^{\log_b a})$
If $a = b^d$, $T(n) = \Theta(n^d \log n)$
If $a > b^d$, $T(n) = \Theta(n^d)$

$$T(n) = 7T(n/2) + cn^2$$

Comparing the Recurrence Relation with the Masters Theorem we get
 $a = 7, b = 2, d = 2$

Using Masters Theorem
 $a > b^d$

Therefore $T(n) = \Theta(n^{\log_b a})$
 $\Rightarrow T(n) = \Theta(n^{\log_2 7})$

General Divide-and-Conquer Recurrence

$$T(n) = aT(n/b) + f(n) \quad \text{where } f(n) = \Theta(n^d), \quad d \geq 0$$

Master Theorem:

- If $a < b^d$, $T(n) = \Theta(n^d)$
- If $a = b^d$, $T(n) = \Theta(n^d \log n)$
- If $a > b^d$, $T(n) = \Theta(n^{\log_b a})$

Note: The same results hold with O instead of Θ .

Examples:

$T(n) = 4T(n/2) + n$	$T(n) = ?$	$\Theta(n^2)$
$T(n) = 4T(n/2) + n^2$	$T(n) = ?$	$\Theta(n^2 \log n)$
$T(n) = 4T(n/2) + n^3$	$T(n) = ?$	$\Theta(n^3)$

Example: Strassen's Matrix Multiplication

$$D = A_1(B_2 - B_4)$$

$$E = A_4(B_3 - B_1)$$

$$F = (A_3 + A_4)B_1$$

$$G = (A_1 + A_2)B_4$$

$$H = (A_3 - A_1)(B_1 + B_2)$$

$$I = (A_2 - A_4)(B_3 + B_4)$$

$$J = (A_1 + A_4)(B_1 + B_4)$$

$$C_1 = E + I + J - G$$

$$C_2 = D + G$$

$$C_3 = E + F$$

$$C_4 = D + H + J - F$$

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} * \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} =$$

$$D = 1(6 - 8) = -2$$

$$E = 4(7 - 5) = 8$$

$$F = (3 + 4)5 = 35$$

$$G = (1 + 2)8 = 24$$

$$H = (3 - 1)(5 + 6) = 22$$

$$I = (2 - 4)(7 + 8) = -30$$

$$J = (1 + 4)(5 + 8) = 65$$

$$C_1 = 8 - 30 + 65 - 24 = 19$$

$$C_2 = -2 + 24 = 22$$

$$C_3 = 8 + 35 = 43$$

$$C_4 = -2 + 22 + 65 - 35 = 50$$

Exercise

Multiply the following two matrix using Strassen's Matrix Multiplication Method. Multiply 2 x 2 matrix conventionally.

5	10	15	20
5	10	15	20
1	2	3	4
5	6	7	8

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

450	500	550	600
450	500	550	600
90	100	110	120
202	228	254	280

Large Integer Multiplication using Divide and Conquer Approach

- There are two ways to perform large integer multiplication using divide and conquer.
- The first method – we call **dumb method** – does not improve the running time.
- Second method – we call **clever approach** – performs better than the traditional approach for integer multiplication.

Dumb Approach:

Let us represent number D as $D = d_{n-1}d_{n-2} \dots d_2d_1d_0$. Each digit d_i is the i^{th} least significant digit in number D . Value of each position is given by,

$d_0 = 10^0$ position

$d_1 = 10^1$ position

$d_2 = 10^2$ position

.

.

$d_{n-1} = 10^{n-1}$ position.

- According to position value,

$$45 = 4*10^1 + 5*10^0$$

$$23 = 2*10^1 + 3*10^0$$

For any real values a, b, c and d,

$$(a + b)*(c + d) = (a*c + a*d + b*c + b*d)$$

$$\text{So, } 45 * 23 = (4*10^1 + 5*10^0) * (2*10^1 + 3*10^0)$$

$$= (4 * 2) * 10^2 + (4*3 + 5*2) * 10^1 + (5*3) * 10^0$$

$$= 800 + 220 + 15$$

$$= 1035$$

- Let's derive formula to multiply two numbers of two digits each. Consider $C = A * B$.
- If we consider $A = a_1a_0$ and $B = b_1b_0$, then
- $C = A * B = c_210^2 + c_110^1 + c_010^0$
- Where,
- $c_2 = a_1 * b_1$
- $c_1 = a_1 * b_0 + a_0 * b_1$
- $c_0 = a_0 * b_0$
- This method does four multiplications, same as conventional method.

Example: Multiply 2345 with 678 using divide and conquer approach.

- **Solution:**

- Size of both operands must be even, so pad zero to the multiplier.
- $A = 2345$ and $B = 0678$
- $A = a_1a_0 = 2345$, hence $a_1 = 23$ and $a_0 = 45$
- $B = b_1b_0 = 0678$, hence $b_1 = 06$ and $b_0 = 78$
- $C = A * B = c_210^4 + c_110^2 + c_010^0$
- $c_2 = a_1 * b_1$
- $= 23 * 06$
- $= 138$

- $c_1 = a_1 * b_0 + a_0 * b_1$
- $= (23 * 78) + (45 * 06)$
- $= 2064$
- $c_0 = a_0 * b_0$
- $= (45 * 78)$
- $= 3510$
- $C = c_2 10^2 + c_1 10^1 + c_0 10^0$
- $= 138 * 10^4 + 2064 * 10^2 + 3510$
- $= 15, 89, 910$

Clever Approach:

- Previous DC approach does four multiplications. Let's reformulate it to reduce numbers of multiplications to three.

- According to dumb approach,
- $c_2 = a_1 * b_1$
- $c_1 = a_1 * b_0 + a_0 * b_1 \dots(1)$
- $c_0 = a_0 * b_0$
- **Second approach:**
- Let us rewrite c_1 as,
- $c_1 = (a_1 + a_0) * (b_1 + b_0) - (c_2 + c_0)$
- $= (a_1 b_1 + a_1 b_0 + a_0 b_1 + a_0 b_0) - (a_1 b_1 + a_0 b_0)$
- $= a_1 * b_0 + a_0 * b_1 \dots(2)$

- For $A = a_1a_0 = 2345$,

- hence, $a_1 = 23$ and $a_0 = 45$ and

- $B = b_1b_0 = 0678$,

- hence, $b_1 = 06$ and $b_0 = 78$

- $c_2 = a_1 * b_1$

- $= 23 * 06$

- $= 138$

- $c_0 = a_0 * b_0$

- $= (45 * 78)$

- $= 3510$

- $c_1 = (a_1 + a_0) * (b_1 + b_0) - (c_2 + c_0)$

- $= (23 + 45) * (06 + 78) - (138 + 3510)$

- $= 68 * 84 - 3648$

- $= 2064$

$$\begin{aligned} C &= c_2 * 10^4 + c_1 * 10^2 + c_0 \\ &= 1380000 + 206400 + 3510 = 15, 89, 910 \end{aligned}$$

Generalization:

- If size of integer is n digit, then we can generalize multiplication as,
- $C = c_2 10^n + c_1 10^{n/2} + c_0$
- Where, $c_2 = a_1 * b_1$
- $c_0 = a_0 * b_0$
- $c_1 = (a_1 + a_0) * (b_1 + b_0) - (c_2 + c_0)$
- This can be solved by applying recursion till n reaches to 1.

Complexity Analysis

- For n digit integer, we have to perform 3 multiplications of integers of size $(n / 2)$. Recurrence equation for this problem is given as,
- $T(n) = 3T(n/2)$, if $n > 1$
- $T(n) = 1$, if $n = 1$
- $T(n) = 3T(n/2) \dots$ **(3)**
- Let us solve this recurrence by an iterative approach. Substitute n by $n/2$ in Equation (3)
- $T(n/2) = 3T(n/4) \dots$ **(4)**
- Put this value in Equation (3),
- $T(n) = 3(3T(n/4)) = 3^2T(n/2^2) \dots$ **(5)**
- Substitute n by $n/2$ in Equation (4)
- $T(n/4) = 3T(n/8)$
- Put this value in Equation (3)
- $T(n) = 3(3^2T(n/8)) = 3^3T(n/2^3) \dots$ **(6)**
- .
- .
- After k iterations,
- $T(n) = 3^kT(n/2^k) \dots$ **(7)**

- Every time number of digits in number reduces by factor 2, so it can go as deep as $\log_2 n$,
- So, $k = \log_2 n \Rightarrow n = 2^k$
- Thus from equation (5), $T(n) = 3^k T(2^k / 2^k)$
- $T(n) = n^{\log_2 3} \times T(1)$ ($\because n^{\log_b a} = a^{\log_b n}$)
- $T(1) = 1$ (Only one multiplication is required to multiply two numbers of digits 1)
- So, $T(n) = n^{\log_2 3} = O(n^{1.58})$

Since $k = \log_2 n$,

$$M(n) = 3^{\log_2 n} = n^{\log_2 3} \approx n^{1.585}.$$

(On the last step, we took advantage of the following property of logarithms:
 $a^{\log_b c} = c^{\log_b a}$.)

Exercise

Solve the following recurrence equations using substitution Method and Master's theorem. In all the examples considered take $t(1)=1$:

$$t(n) = 10t(n/3) + 11n, \quad n \geq 3 \text{ and a power of } 3.$$

$$t(n) = 128t(n/2) + 6n^8, \quad n \geq 2 \text{ and a power of } 2.$$